Clustering Customer based on geospatial data using DBSCAN.

Companies often have a lot of data related to their customers, one of which is geospatial/geolocation data, which consist of *latitude and longitude* information of the customer address. Proper storage and utilization of such data can give company meaning insights on costumers, increase customer loyality and gain upper hand among its competitors.

One of a company can attract and build up loyal customer is by focusing on building its core-compentency. Say we have company that wants to prioritize *customer service*. One way of gaining meaningful insights on the customers is to cluster customer based on their locations, then provide them fast and consistent customer service. For this the company need to gather geospatial data of customers, cluster customer based on locations and spend resources on opening service locations at the *center* of those clusters (shortest distance to all locations).

Learning Objectives

In this programming material, you are learning to doing just that. We will

1. **Examine the synthetic geospatial data** of a arbitary company. (Visualization)
2. **Perform density-based clustering, DBSCAN** using `sklearn` on the data with optimal parameter estimation.
3. **Compute center/centroids** of those clusters.

Importing required libraries

In [6]:

```
# Basic required libraries
import numpy as np
import matplotlib.pyplot as plt
```

In [7]:

```
# Setting parameters for plots
plt.rcParams.update({
    'figure.figsize':(12,10),
    'font.size':14,
})
```

Loading the Data Set

The dataset we are about to work on is created by us and is *not a real dataset*. The data is the geospatial location of customer of a private Nepal based company (name not important) centered at its capital Kathmandu. So, most of its customer are from Kathmandu and Lalitpur district.

Let's load dataset present in a csv file using Pandas `read_csv()` function and store it in variable named `geo_data` .

In [8]:

```
# Importing Pandas library
import pandas as pd

# Loading the dataset into Pandas Dataframe
geo_data = pd.read_csv('https://storage.googleapis.com/codehub-data/1-lv2-9-3P-customer_geo
```

Let's print shape and few of the initial data samples.

In [9]:

```
# Shape of datase
print(f'Shape of the dataset:{geo_data.shape}')

# Displaying head values
geo_data.head()
```

```
Shape of the dataset:(1003, 2)
```

Out[9]:

	latitude	longitude
0	27.689118	85.333783
1	27.693842	85.351062
2	27.705457	85.345629
3	27.704850	85.346461
4	27.704807	85.347880

The dataset consists of 1003 data samples with 2 features, the longitude and latitude of the customer's permanent address.

Visualizing the Geospatial Data

Since geospatial data only consists of two features, latitude and longitude, its really easy to visualize. One of the libraries that allows visualization of geospatial data is `folium`.

First, we generate a map using `.Map()` function and store it in `ktm_map` with parameters:

- `location` : List consisting of latitude and longitude to center the map. Using location of Kathmandu with latitude: 27.69 and longitude: 85.35.
- `tiles` : Type of Map to use. Using `cartodbpositron`
- `zoom_start` : Initial zoom level of the map, Set to 12.

In [10]:

```
# Import folium for visualization of geospatial data
import folium

# Create a map
ktm_map = folium.Map(
        location  = [27.69, 85.35],
        tiles='cartodbpositron',
        zoom_start=12)
```

Next, we use folium's `CircleMarker` function that creates marker provided location. We can easily apply the function to every data point of the dataset using pandas `apply` function and later add the markers to `ktm_map` .

The parameters passed in `CircleMarker` function are:

- location: List consisting of latitude and longitude of locations.
- radius: radius of the circle marker.

In [11]:

```
# Add markers at the location of data points
geo_data.apply(lambda x: folium.CircleMarker(location=[x['latitude'],x['longitude']] \
                                             , radius=2).add_to(ktm_map),axis=1)
```

Out[11]:

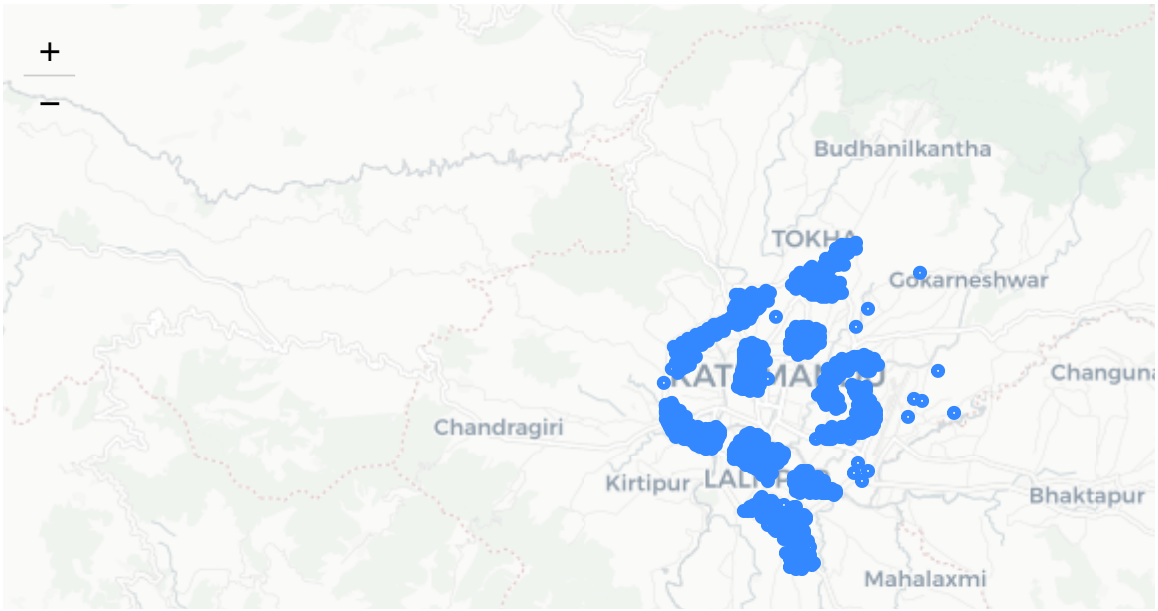
```
0       <folium.vector_layers.CircleMarker object at 0...
1       <folium.vector_layers.CircleMarker object at 0...
2       <folium.vector_layers.CircleMarker object at 0...
3       <folium.vector_layers.CircleMarker object at 0...
4       <folium.vector_layers.CircleMarker object at 0...
                              ...
998     <folium.vector_layers.CircleMarker object at 0...
999     <folium.vector_layers.CircleMarker object at 0...
1000    <folium.vector_layers.CircleMarker object at 0...
1001    <folium.vector_layers.CircleMarker object at 0...
1002    <folium.vector_layers.CircleMarker object at 0...
Length: 1003, dtype: object
```

Finally lets visualize the map with markers at the location of data.

In [12]:

```
# Visualizing the map with markers
ktm_map
```

Out[12]:



Looking at the visualization above, there are 10 distinct clusters with similar density that are separated by less dense regions. However, some the clusters are non-convex in nature. Finally, we can see some outlier/noise points present in the dataset.

Haversine distance

Before starting the cluster, we require knowledge on calculation of distance between two data using pair of latitude and longitude information. Note: Even though latitudes line are evenly spaced, the distance are distroted far from the equator covering lower distance at poles than at equator.

Calculation of geodetic distance between two points on Earth using geospatial data is done using **Haversine (or great circle) distance**. Haversine distance is the angular distance between two points on the surface of a sphere(Earth is assumed perfectly spherical). The distance is taken in **radians** with the first value being latitude and second being longitude. The haversine distance between two points is given as:

$$D(x, y) = 2\arcsin[\sqrt{\sin^2((x_1 - y_1)/2) + \cos(x_1)\cos(y_1)\sin^2((x_2 - y_2)/2)}]$$

where, x - latitude value, y - latitude value. Both of them must be in radians.

Clustering using K-Means

K-Means is a really popular algorithm for clustering data points. However, K-means clustering tends to minimize the variance not the geodetic distance between points. In short, K-means would work for latitude-longitude data but not optimally. Also, the non-convex clusters and outliers result in poor performance of K-means in current task. Nevertheless, let's see how K-means performs clustering on our synthetic geospatial data.

The parameters required to initialize K-means algorithm are:

- `n_cluster` : 10, as the number of clusters are ten.
- `init` : `'random'` , initializing the centriod as random points.

In [13]:

```
# Import K-means
from sklearn.cluster import KMeans

# Initial K-means
kmeans = KMeans(n_clusters=10, random_state=0)
```

Let's fit the dataset and store the cluster labels in the `geo_data` dataframe as new column labeled `cluster kmeans`

In [14]:

```
# Cluster Assignment Kmeans
cluster_kmeans = kmeans.fit(geo_data)

# Adding cluster labels to the dataframe
geo_data['cluster kmeans'] = cluster_kmeans.labels_
```

Visualization of clustering using Kmeans

Finally, let's visualize the clustering conducted using Kmeans.

- First, we specify the colors of the data points per cluster in list named `cluster_colors` . Eleven colors; ten for cluster, one for outlier. (Black for outlier).
- Then, use the `CircleMarker` function as we did in first visualization but with additional `color` parameter, where the colors are selected using the `cluster kmeans` column of the `geo_data` dataframe.

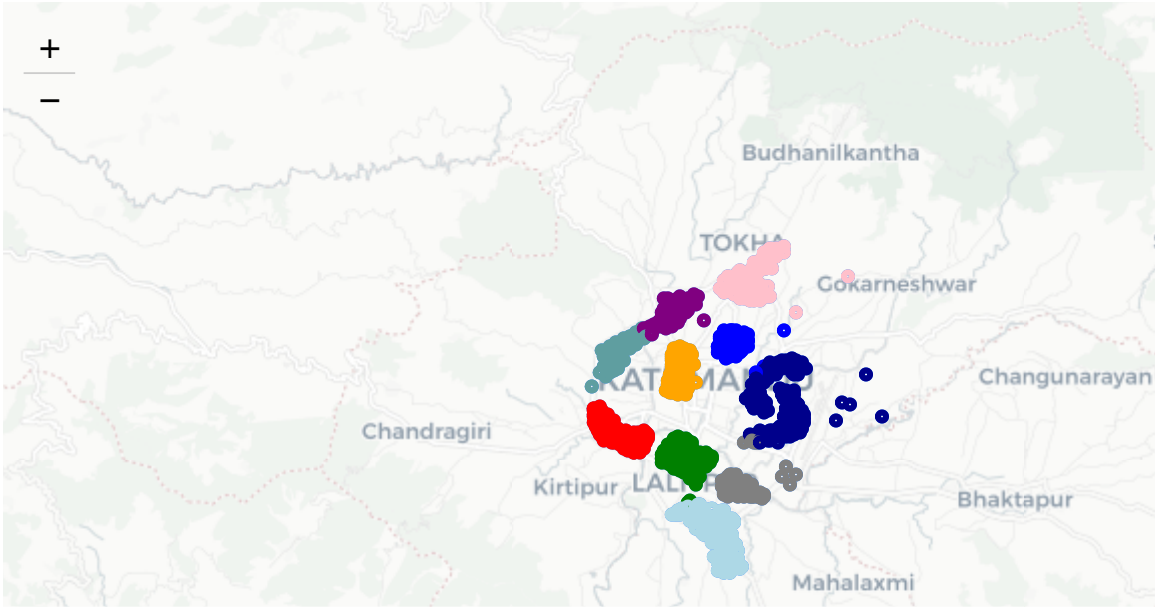
In [15]:

```
# Colors for visualizing clusters
cluster_colors = [
    'red',
    'blue',
    'gray',
    'orange',
    'green',
    'darkblue',
    'lightblue',
    'purple',
    'pink',
    'cadetblue',
    'black'
]

# Adding markers for clusters
geo_data.apply(lambda x: folium.CircleMarker(location=[x['latitude'],x['longitude']], radiu
                                             color=cluster_colors[x['cluster kmeans'].astyp
                                               .add_to(ktm_map),axis=1)

# Visualizing Clustering using Kmeans
ktm_map
```

Out[15]:



K-means clustering didn't perform well on our geospatial data. Look at the 2 C shaped clusters on the right is assigned to the same cluster, while the single elongated cluster on the top left is separated into two. Other clusters are correctly identified by K-means with minor errors. K-means cannot segregate the outliers, consequently assigning them to clusters.

Finding the centriods for the above cluster assignment would be unsatisfactory due to improper detection of clusters and assigment of outliers to clusters. Now that we saw mediocre results from K-means clustering. Let's use DBSCAN to cluster the data-points.

Clustering using DBSCAN

DBSCAN is a density-based clustering algorithm. It can easily detect outliers and cluster non-convex dataset. DBSCAN can use any pairwise distance metric, such as haversine distance to compute the distance between any pair of point (neighborhood). DBSCAN seem perfect algorithm for our task.

Let's import the required libraries for implementation of DBSCAN using `sklearn` .

In [16]:

```
# Importing required libraries
from sklearn.cluster import DBSCAN
```

Converting latitude and longitude values from degrees to radians

Before implementing DBSCAN, we require conversion of geospatial latitude-longitude from degrees to radians.

- First, let's convert the `geo_data` into numpy array for easier manipulation.
- Then, convert the numpy array containing the latitude-longitude (coordinate in degrees) data to radians using `np.radians` function.

In [17]:

```
# Converting dataframe in numpy array
coordinates = np.array(geo_data[['latitude', 'longitude']])

# Converting coordinates to radians
coordinates_in_radians = np.radians(coordinates)
```

Best value of ϵ

Now we can implement DBSCAN algorithm. As you learned in theory, DBSCAN is very sensitive to input parameter, the clustering depends on the initial set of parameters. Authors in the original paper presented an interactive method to select best value of ϵ which uses **Sorted K-distance graph**.

Steps of obtaining Sorted K-distance graph:

- Select a value of "K".
- Select a data point and compute its distance to its K^{th} nearest neighbor.
- Sort the data points in descending order according to the distance to its K^{th} nearest neighbor.

Using Sorted K-distance graph, we can find the threshold value of ϵ that generates the thinnest(least dense) cluster given the value of MinPts parameter. The threshold value is the first point in the first valley of k-sorted graph. The x-axis represents ordering of points, while the y-axis represents the distance of respective data point to its K^{th} neighbor.

To compute the nearest neighbor distance, let's use `NearestNeighbors` estimator provided by sklearn with harvesian distance as distance metric and fit the latitude-longitude data in radian, i.e. `coordinates_in_radians` .

Suppose the company decides that they require *atleast 15 customers* in close proximity as a requirement of opening customer care office at that location. So, setting `min_pts=15` , let's compute the sorted k-distance graph.

First, we fit the `NearestNeighbor` estimator with required parameters, store the object in a variable, and then compute distance using `kneighbors` that output distance from 1 to k^{th} and plus their indices.

In [18]:

```
# Import
from sklearn.neighbors import NearestNeighbors

# sorted k-distance graph

# MinPts parameters
min_pts = 15

# Computing the neighbors
n_neighbors = NearestNeighbors(n_neighbors=min_pts, metric='haversine').fit(coordinates_in_
distances, indices = n_neighbors.kneighbors(coordinates_in_radians)
```

Measuring distance in Kilometres in more intuitive for us humans, same goes for the ϵ parameter. Since, the haversine distance is in radian, we need to convert its value from radians to Kms. For this let's store the conversion of radians to kilometrics in variable named `rads_to_kms=6371` , because 1 rad = 6371 Km (radius of Earth).

In [19]:

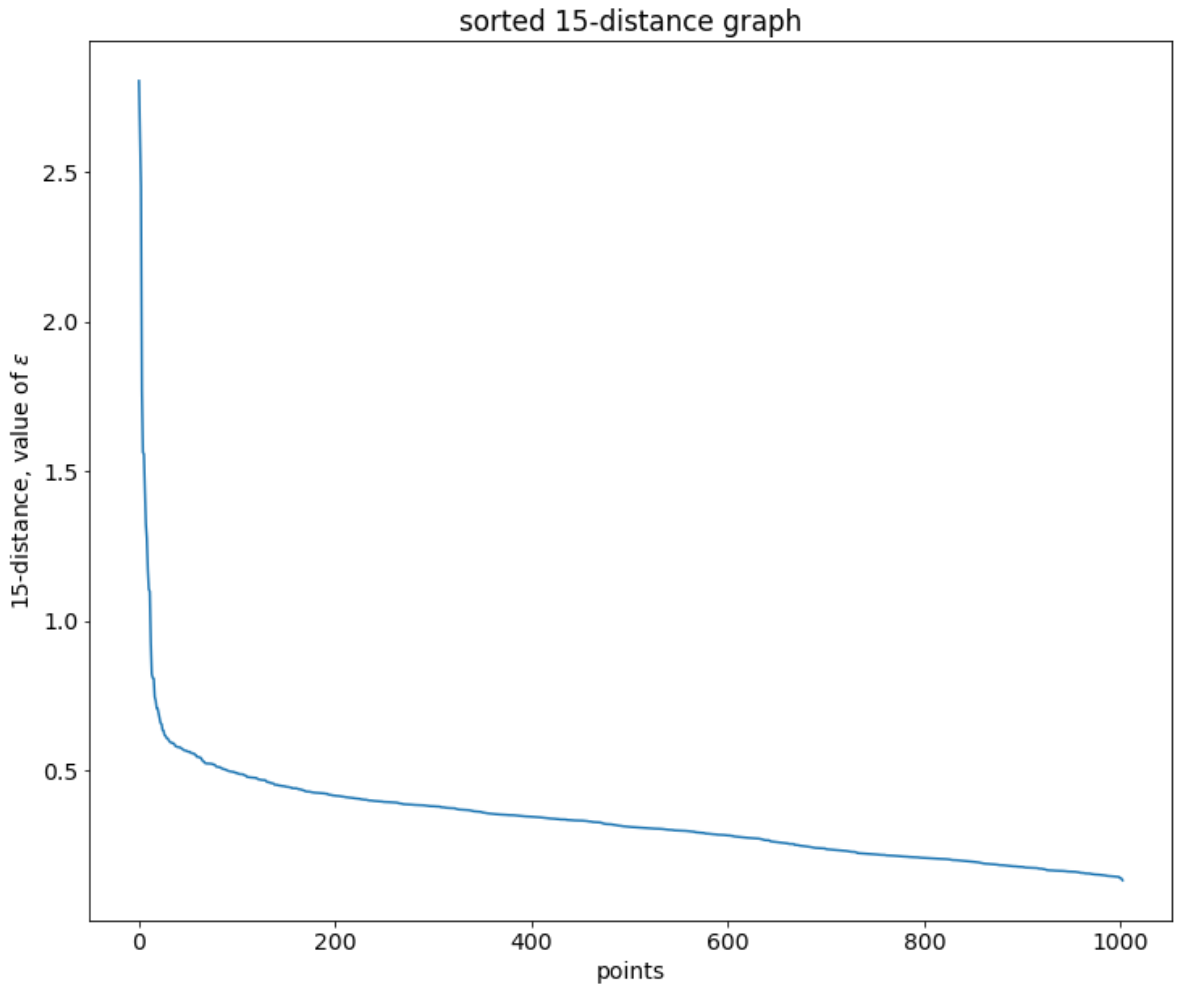
```
# Conversion from radians to Kms
kms_per_radian = 6371.0088
```

Then plot the sorted k-distance graph.

In [20]:

```
# Ordering distance to kth-neighbor
distance_to_k = sorted(distances[:,min_pts-1]*kms_per_radian, reverse=True)

# Plotting the sorted k-distance graph
plt.plot(indices[:,0], distance_to_k)
plt.title('sorted 15-distance graph')
plt.xlabel('points')
plt.ylabel(r'15-distance, value of $\epsilon$')
plt.show()
```



The 1st valley occurs at about, $\epsilon = 0.5\text{km}$. That is the value we are using for ϵ in our implementation.

Implementation of DBSCAN clustering

We start by converting the value of optimal to radians. Then initialize DBSCAN onto `dbscan` variable using parameters:

- `eps` : ϵ value, size of neighborhood.
- `min_samples` : MinPts, smallest number of data points in neighbor of a point to be considered as core point.
- `metric` : `haversine` to compute the harvesine distance.

Then, we fit `coordinates_in_radians` data to DBSCAN. Get the cluster label in `.labels_` property,

In [21]:

```
# Converting kms into radians
epsilon = 0.5/ kms_per_radian

# Initialization of DBSCAN
dbscan = DBSCAN(eps=epsilon, min_samples=15, metric='haversine')

# Fitting the geospatial data
dbscan.fit(coordinates_in_radians)

# Printing number of clustres
print('Number of clusters found: ',len(np.unique(dbscan.labels_))-1)
```

```
Number of clusters found:  10
```

DBSCAN found 10 clusters in the dataset and the outlier as well, let's store the cluster assignement and visualize the clustering performed by dbscan.

In [22]:

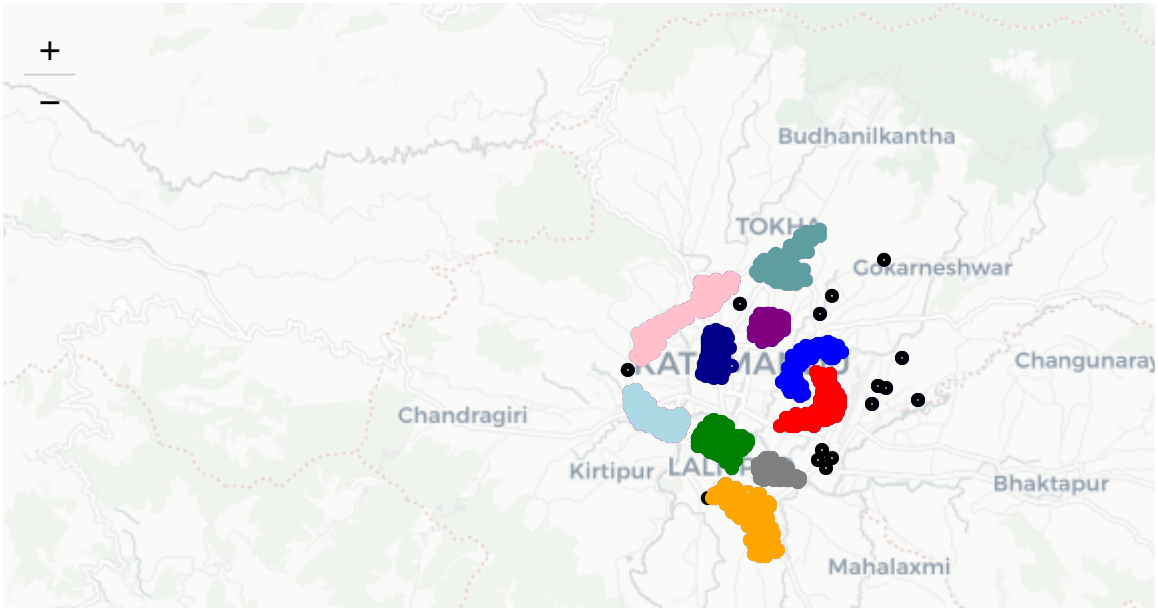
```
# Storing cluster assignment to dataframe
geo_data['dbscan_cluster']= dbscan.labels_
```

In [23]:

```
# Visualization of cluster assignment
geo_data.apply(lambda x: folium.CircleMarker(location=[x['latitude'],x['longitude']], radiu
                                             color=cluster_colors[x['dbscan_cluster'].astyp

ktm_map
```

Out[23]:



Most of the problem faced using K-means clustering is solved by DBSCAN. Non-convex data is properly identified and clustered with detection of outliers. This is the power of density-based clustering. Finally let's remove the outlier from the dataset and compute the centroids of each cluster, which are the locations to open customer service centers.

In [24]:

```
# Removing the outlier points
geo_data_without_outlier = geo_data.loc[geo_data['dbscan_cluster'] != -1] \
                                        [['latitude', 'longitude', 'dbscan_cluster']]
```

Now, store the centroids/means of each cluster, using `groupby` and `apply` functions that groups the data based on cluster assignment(`dbscan_cluster`) and compute the mean of each group, respectively.

In [25]:

```
# Computing centroids of cluster
centroids = geo_data_without_outlier.groupby('dbscan_cluster')[['latitude', 'longitude']\
                                                               ].apply(lambda x: x.mean())

# Reseting the index of data frame
centroids = centroids.reset_index()
```

Last task remains after computation of the centroids of each cluster, visualizing where the centroids lie. Let's repeat the visualization using folium library, but let's create a new map named `ktm_map_centroids` and add the centroids.

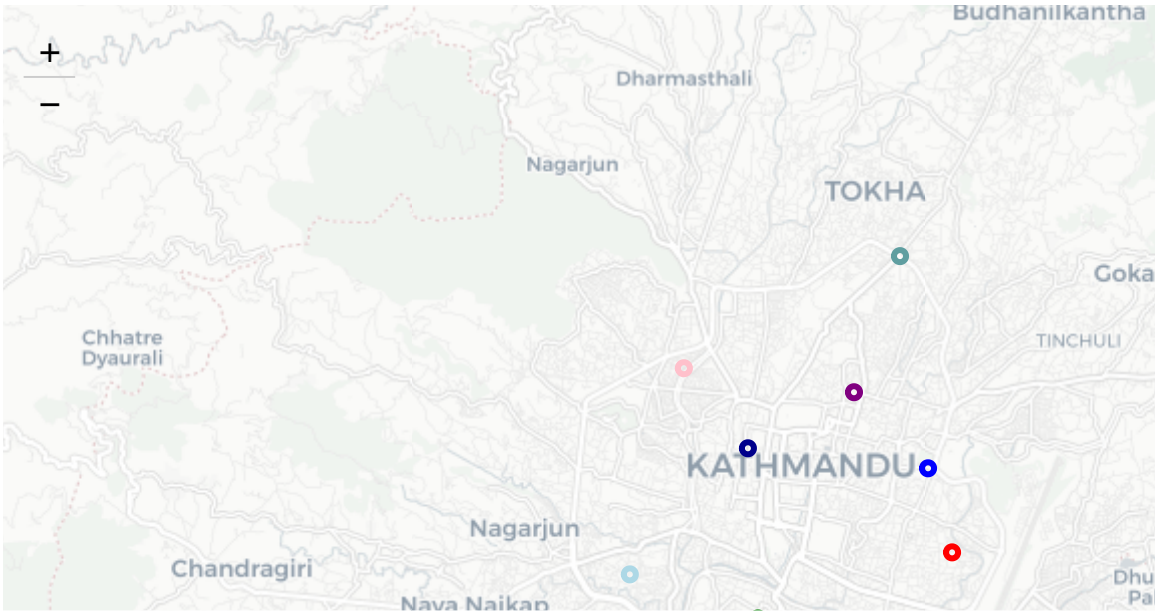
In [26]:

```
# Visualization of cluster assignment
# Creating map
ktm_map_centroids = folium.Map(
        location  = [27.69, 85.35],
        tiles='cartodbpositron',
        zoom_start=12)

centroids.apply(lambda x: folium.CircleMarker(location=[x['latitude'],x['longitude']], radi
                                              color=cluster_colors[x['dbscan_cluster'].astyp
                                             .add_to(ktm_map_centroids),axis=1)

# Visualizing Centroids
ktm_map_centroids
```

Out[26]:



These marked points are the best locations for opening customer service outlets, which are the centroid value of each cluster.

Customer clustering based of geospatial data is one of the use of clustering. Additional extra data such as average expenditure, family status can be used and linked with geospatial data for targeting specific products at a particular group of people.

Altogether in this programming, you learnt to visualize customer geospatial data, cluster the customer using DBSCAN that identified 10 cluster/group of customers based on the geospatial data, and finally compute the centroids of clustered groups.

In [ ]:

```
```